Prepoznavanje zdravstvenog stanja pčela na osnovu fotografije

Transfer learning, poređenje arhitektura i integracija u informacioni sistem „Košnica“

Medžid Jašarović

07.10.2026.

Sadržaj

- Sažetak
- Uvod
  - Cilj rada
  - Istraživačka pitanja
  - Tok rada i ključni nalaz
  - Struktura izveštaja
- Pregled javnih skupova podataka
  - Kriterijumi procene
  - Pronađeni skupovi
  - Izbor
- Prvi eksperiment: BeeImage
  - Postavka
  - Rezultati na test skupu
  - Problem otkriven ručnim testom
  - Uzrok: oznaka opisuje snimak, ne pčelu
  - Dokaz: test prečice
  - Pouke za glavni eksperiment
- Skup podataka VarroaDataset
  - Raspodela po skupovima i klasama
  - Podela po snimku
  - Grinja na slici
- Transfer learning — pristup i primena
  - Zašto se mreža ne trenira od nule
  - Tri arhitekture
  - Primenjeni postupak
  - Priprema slike (*letterbox*)
  - Augmentacija
  - Disbalans klasa
  - Isti uslovi za sve tri arhitekture
- Izbor okvira: PyTorch ili TensorFlow/Keras

- Eksperimentalni rezultati
  - Izbor protokola treninga
  - Uslovi eksperimenta
  - Poređenje arhitektura na test skupu
  - Metrike po klasi
  - Matrice konfuzije
  - Krive učenja
  - Rezultat po test snimku
  - Grinja vidljiva i delimično skrivena
- Da li model gleda grinju?
  - Provera 1: brisanje grinje
  - Provera 2: samo pozadina
  - Provera 3: Grad-CAM
- Test van skupa za trening
  - BeeDataset — druga kamera, druga podloga
  - Terenske fotografije
- Izbor modela
  - Kriterijumi
  - Svi kriterijumi na jednom mestu
  - Odluka: ResNet50
  - Rezerve
  - Poređenje sa prvim eksperimentom
- Integracija u informacioni sistem „Košnica“
  - Postojeći sistem
  - Arhitektura rešenja
  - Zašto zaseban Python servis
  - Python servis (FastAPI)
  - Backend (ASP.NET Core)
  - Frontend (Angular)
  - Zamena modela
- Ograničenja i dalji rad
  - Ograničenja skupa podataka
  - Ograničenja eksperimenta
  - Ograničenja integracije
  - Dalji rad
- Prilog: reprodukcija rezultata
  - Okruženje
  - Glavni eksperiment (VarroaDataset)
  - Prvi eksperiment (BeeImage)
  - Notebook-ovi
  - Pokretanje aplikacije

Sažetak

Zadatak. Prepoznavanje zaraze pčele grinjom *Varroa destructor* sa fotografije pojedinačne pčele, transfer learning-om, uz poređenje tri arhitekture i integraciju najboljeg modela u informacioni sistem „Košnica“.

Izabrani model: ResNet50, treniran na skupu VarroaDataset (13.509 slika, oznaka po pčeli). Na test skupu od 3.408 slika iz dva snimka koje model nije video tokom treninga:

Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	Recall — varoa	ROC AUC
0,9310	0,9152	0,9117	0,9134	0,8684	0,9753

- Kada se na zaraženoj pčeli grinja prekrije, model i dalje kaže „varoa“ u 3,9% slučajeva, a kada se prekrije isto toliko mesta na drugom delu pčele — u 98,2%. Odluka modela zavisi od grinje.
- Na nezavisnom skupu BeeDataset (druga kamera i podloga, 7.490 slika) ROC AUC je 0,8694, a recall na varoi 0,3049.
- Inferencija u produkcionom runtime-u (ONNX, CPU) traje 11,9 ms po slici; fajl modela ima 94,2 MB.

Prvi eksperiment (skup BeeImage) dao je tačnost 97% na test skupu, ali model nije prepoznavao bolest nego pozadinu snimka; to je otkriveno ručnim testom i dokazano posebnim eksperimentom. Pouke iz tog eksperimenta su ugrađene u glavni eksperiment.

Integracija. Model se izvršava u Python/FastAPI servisu (ONNX Runtime). ASP.NET backend prosleđuje fotografiju servisu, čuva sliku i rezultat u istoriji pregleda košnice; Angular aplikacija ima novi tab „Zdravlje pčela“ u modalu košnice.

Uvod

Cilj rada

Cilj rada je da se u postojeći informacioni sistem „Košnica“ — veb aplikaciju za vođenje pčelinjaka, košnica, intervencija i prinosa — doda funkcionalnost **procene zdravstvenog stanja pčele na osnovu fotografije**. Korisnik učita fotografiju pčele, sistem je prosledi modelu mašinskog učenja, a model vrati procenu zdravstvenog stanja i verovatnoću te procene. Rezultat se vezuje za konkretnu košnicu i čuva u istoriji pregleda.

Model se razvija **transfer learning**-om: umesto da se mreža trenira od nule, koriste se mreže prethodno istrenirane na velikom opštem skupu slika, koje se prilagođavaju ovom zadatku. Upoređuju se tri arhitekture — **MobileNetV3, ResNet50 i EfficientNetB0** — pod istim uslovima, po metrikama accuracy, precision, recall, F1-score i matrici konfuzije, i najbolja se integriše u sistem.

Istraživačka pitanja

Rad odgovara na sledeća pitanja:

1. Koji kvalitetni javni skupovi označenih slika pčela postoje, koje klase sadrže i da li odgovaraju ovom problemu?
2. Kako se transfer learning praktično primenjuje sa MobileNet, ResNet i EfficientNet modelima?
3. Da li je za ovaj zadatak pogodniji PyTorch ili TensorFlow/Keras?
4. Koja od tri arhitekture daje najbolje rezultate, i po kojim kriterijumima?
5. Kako se istrenirani model povezuje sa postojećim ASP.NET sistemom?

Tok rada i ključni nalaz

Rad je izveden u dva eksperimenta. Prvi je koristio skup **BeeImage**, najpoznatiji javni skup slika pčela sa oznakama zdravlja. Modeli su na njegovom test skupu postigli tačnost do 97%, ali je ručno testiranje u aplikaciji pokazalo da model fotografiju pčele sa dve jasno vidljive grinje procenjuje kao zdravu sa 99,97%. Analiza je pokazala da oznake u tom skupu opisuju snimak, a ne pčelu, i da je mreža naučila da prepoznaje pozadinu snimka umesto bolesti — pojava poznata kao *shortcut learning*.

Drugi, glavni eksperiment koristi skup **VarroaDataset**, u kome je svaka pčela označena posebno, na osnovu vidljive grinje *Varroa destructor*. Uz klasične metrike, za svaki model je posebnim testovima provereno **da li zaista gleda grinju** — što je pouka prvog eksperimenta.

Struktura izveštaja

Poglavlje	Sadržaj	Pitanje
Pregled javnih skupova podataka	pet skupova, kriterijumi, izbor	1
Prvi eksperiment: BeeImage	rezultati i zašto model ne radi	1
Skup podataka VarroaDataset	analiza izabranog skupa	1
Transfer learning — pristup i primena	postupak korak po korak	2
Izbor okvira: PyTorch ili TensorFlow/Keras	obrazloženje	3
Eksperimentalni rezultati	poređenje tri arhitekture	4
Da li model gleda grinju?	provere ponašanja modela	4
Test van skupa za trening	drugi skup i terenske fotografije	4
Izbor modela	odluka i obrazloženje	4
Integracija u sistem „Košnica“	arhitektura i implementacija	5
Ograničenja i dalji rad		

Pregled javnih skupova podataka

Za prepoznavanje zdravstvenog stanja pčele sa fotografije potreban je skup slika u kome je oznaka zdravlja **dodeljena svakoj pčeli posebno, na osnovu onoga što se na njoj vidi**. Ovaj uslov se pokazao presudnim: prvi korišćeni skup ga ne ispunjava, i model treniran na njemu nije naučio da prepozna bolest (poglavlje „Prvi eksperiment: BeeImage“).

Kriterijumi procene

Svaki skup je procenjen po pet kriterijuma:

1. **Oznaka po pčeli** — da li je oznaka dodeljena pojedinačnoj pčeli na osnovu vidljivog obeležja, ili celom snimku/košnici.
2. **Ista pozadina za sve klase** — ako se klase razlikuju po pozadini, mreža može naučiti pozadinu umesto bolesti.
3. **Mogućnost poštene podele** — da li postoji podatak o snimku/sesiji, tako da test skup sadrži snimke koje model nije video.
4. **Zadatak** — klasifikacija slike (ono što ovaj rad radi) ili detekcija objekata (traži drugačiji tip modela).
5. **Dostupnost i licenca.**

Pronađeni skupovi

Skup	Slika	Klase	Oznaka po pčeli	Ista pozadina	Podela po snimku	Zadatak
BeeImage (Kaggle, Yang 2018)	5.172	6 zdravstvenih stanja košnice	ne — po snimku	ne	ne	klasifikacija
VarroaDataset (Schurischuster i Kampel, TU Wien 2020)	13.509	zdrava / varoa	**da**, sa ručno označenom grinjom	**da**	**da**, zvanična	klasifikacija
BeeDataset (Hickert, BeeAlarmed 2021)	7.490	varoa, polen, hlađenje, osa (više oznaka)	da	da (zelena podloga)	ne	klasifikacija
Bilik i sar., *Sensors* 2021	600	zdrava / zaražena pčela, grinja	da	ne	—	detekcija objekata
Roboflow Universe (više skupova)	od nekoliko stotina do par hiljada	grinja, pčela	da	ne	—	detekcija objekata

BeeImage — *The BeeImage Dataset: Annotated Honey Bee Images* (Kaggle)

Isečci pojedinačnih pčela iz video snimaka na ulazu u košnice na osam lokacija u SAD. Kolona `health` ima šest vrednosti: *healthy*, *few varroa, hive beetles*, *Varroa, Small Hive Beetles*, *ant problems*, *hive being*

robbed, *missing queen*.

Na prvi pogled odgovara problemu — ima i varou i druga stanja. Detaljna analiza je pokazala da oznaka opisuje **stanje košnice u trenutku snimanja**, a ne pojedinačnu pčelu: sve pčele sa istog snimka nose istu oznaku, i na većini isečaka iz klase „varoa“ grinja se ne vidi. Svaki snimak pripada tačno jednoj klasi, a klasa varoe potiče iz svega pet snimaka (89% iz dva). Isečci su sitni (srednja veličina 61×61 piksel), pa je grinja i kad postoji veličine par piksela.

Procena: nije pogodan za prepoznavanje bolesti sa fotografije pojedinačne pčele. Korišćen je u prvom eksperimentu; zašto ne radi, opisano je u posebnom poglavlju.

VarroaDataset (TU Wien) — izabran

Schurischuster S., Kampel M. *VarroaDataset*, Zenodo, 2020, doi: 10.5281/zenodo.4085044, licenca CC BY 4.0.

13.509 slika pojedinačnih pčela rezolucije 160×280 piksela, snimljenih odozgo u kontrolisanim uslovima, na istoj sivoj podlozi. Pčela je označena kao zaražena ako se na njoj **vidi** grinja *Varroa destructor*, i svaka vidljiva grinja ima ručno nacrtan pravougaonik (*bounding box*).

- 9.562 zdrave i 3.947 zaraženih pčela (odnos 2,4 : 1)
- 13 video snimaka iz sedam dana (avgust–oktobar 2017), **svaki snimak sadrži i zdrave i zaražene pčele** (od 20% do 100% zaraženih) — pozadina zato ne može da otkrije klasu
- zvanična podela na train/val/test od strane autora ide **po snimku**: nijedan snimak se ne pojavljuje u dva skupa
- grinja je na slici veličine oko 32 piksela i jasno vidljiva

Dokumentacija opisuje oznake 0 (zdrava) i 1 (zaražena). U fajlu postoji i nedokumentovana oznaka 3 (864 slike). Zvanična statistika autora (3.947 zaraženih) jednaka je zbiru oznaka 1 i 3, sve slike sa oznakom 3 imaju označenu grinju, a vizuelno su to slučajevi kada je grinja delimično skrivena (ispod krila, sa strane). U ovom radu se oznaka 3 tretira kao zaražena, a rezultat se dodatno prikazuje odvojeno za oznake 1 i 3.

Procena: pogodan — ispunjava sva tri ključna uslova. Ograničenje je da ima samo dve klase (zdrava/varoa) i da su sve slike snimljene u istim uslovima, što otvara pitanje rada na fotografijama drugačijeg porekla. To se u radu posebno meri (poglavlje „Test van skupa za trening“).

BeeDataset (BeeAlarmed)

Hickert F. *Dataset for a camera based bee-hive monitoring*, BeeAlarmed, 2021, licenca GPL. Dostupan preko TensorFlow Datasets (`bee_dataset`).

7.490 slika pčela na ulazu u košnicu, snimljenih odozgo kroz staklo, na zelenoj podlozi, sa četiri nezavisne oznake po slici: varoa, polen, hlađenje košnice krilima, osa. Oznake su dodeljene ručno, po pčeli.

Procena: pogodan kao nezavisan test skup. Snimljen je drugom kamerom, u drugoj zemlji, na drugoj podlozi. Model treniran na VarroaDataset-u, testiran na BeeDataset-u, daje meru prenosivosti na drugačije snimanje — što je bliže stvarnoj upotrebi od testa na istom skupu. Nema podatak o snimku, pa nije pogodan kao primarni skup za trening.

Skupovi za detekciju objekata

Bilik S. i sar. (*Visual Diagnosis of the Varroa Destructor Parasitic Mite in Honeybees Using Object Detector Techniques*, Sensors 21(8), 2021) i više skupova na platformi Roboflow Universe sadrže fotografije sa označenim položajem grinja i pčela. Namenjeni su modelima za detekciju objekata (YOLO, SSD), koji na slici pronalaze i lokalizuju svaku grinju. To je drugačiji zadatak od klasifikacije kojom se bavi ovaj rad. Skupovi na Roboflow-u su različitog kvaliteta, sa nejasnim poreklom slika i licencama. **Procena: van obima**, ali relevantni za dalji rad.

Izbor

Za glavni eksperiment izabran je **VarroaDataset**, a **BeeDataset** se koristi kao nezavisan test skup. Klasifikacija je time svedena na dve klase — zdrava i varoa — što je opravdano jer je varoa najznačajniji zdravstveni problem pčela, a nijedan pronađeni javni skup nema pouzdane oznake po pčeli za druga stanja.

Prvi eksperiment: BeeImage

Postavka

Prvi eksperiment je izveden nad skupom BeeImage (5.172 slike), sa svim šest originalnih klasa i sa klasama grupisanim u tri: `zdrava`, `varoa_grinje` (obe klase sa varoom) i `drugi_problem` (mravi, pljačka, bez matice). Slike su nasumično, stratifikovano po klasi, podeljene u odnosu 70/15/15. Trenirane su sve tri arhitekture istim postupkom transfer learning-a kao u glavnom eksperimentu.

Rezultati na test skupu

Arhitektura	Accuracy	Precision	Recall	F1 (macro)	Recall varoa
MobileNetV3-Large	0,9704	0,9529	0,9767	0,9643	0,9747
ResNet50	0,9716	0,9557	0,9698	0,9625	0,9620
EfficientNetB0	0,9111	0,8701	0,9278	0,8954	0,9241

Tri grupisane klase, test skup od 776 slika, nasumična podela.

Po svim metrikama rezultati su izuzetno dobri. Najbolji model (MobileNetV3-Large) izvezen je i integrisan u sistem.

Problem otkriven ručnim testom

Pri ručnom testiranju u aplikaciji, fotografija pčele na saću sa dve jasno vidljive grinje procenjena je kao **zdrava sa 99,97%**.

Fotografija koja je otkrila problem: pčela sa dve vidljive grinje, model kaže „zdrava“ 99,97%

Uzrok: oznaka opisuje snimak, ne pčelu

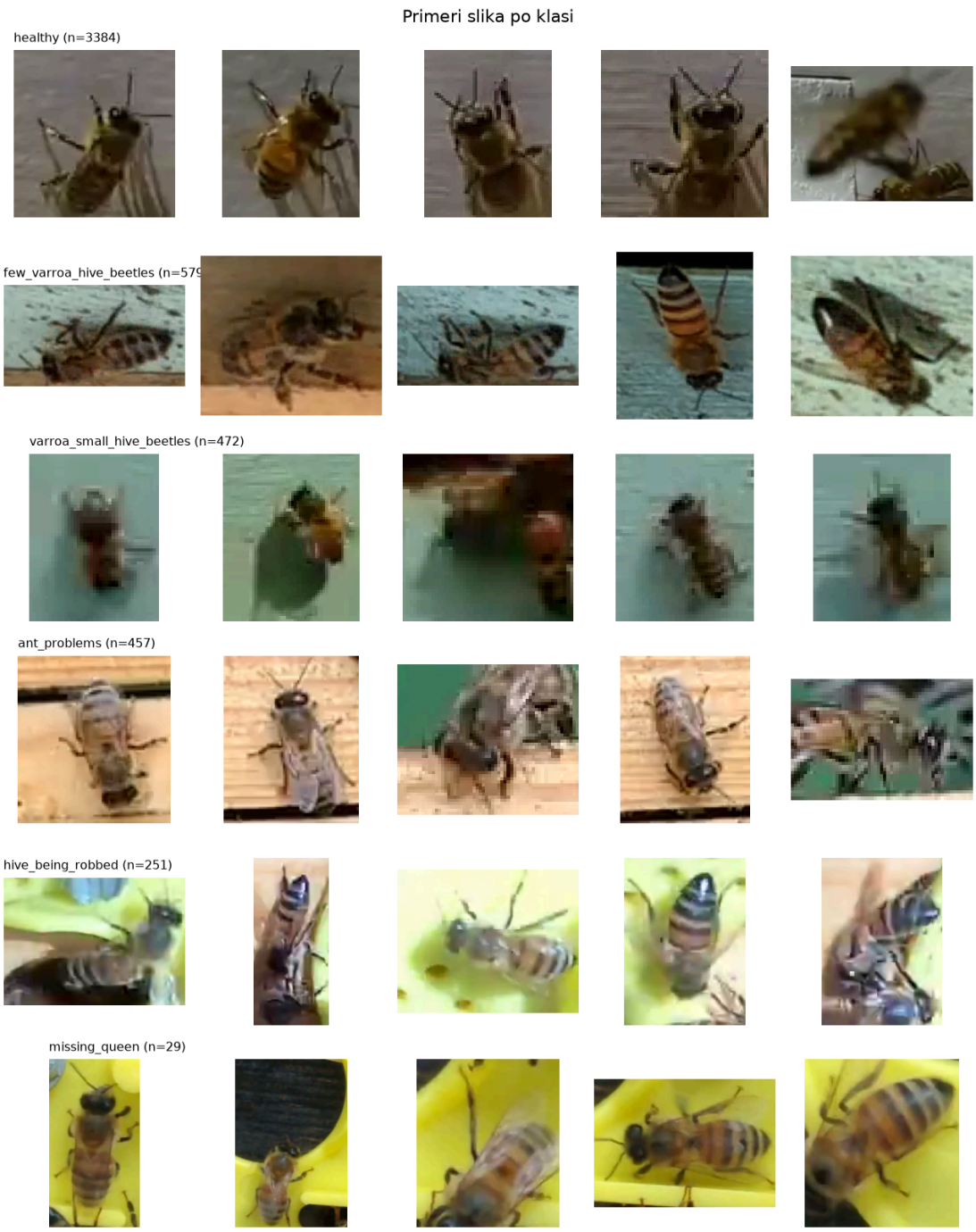
Analiza skupa pokazala je tri svojstva koja zajedno objašnjavaju ponašanje modela:

1. **Oznaka je dodeljena celom snimku.** BeeImage je nastao isecanjem pčela iz video snimaka na ulazu u košnicu; svaka pčela dobija oznaku stanja košnice. Na većini isečaka iz klase „varoa“

grinja se ne vidi.
2. **Svaki snimak pripada tačno jednoj klasi.** Postoji 45 snimaka (prepoznatih po prefiksu imena fajla) i nijedan ne prelazi granicu klase. Klasa varoe potiče iz svega pet snimaka, a 89% njenih slika iz dva.
3. **Nasumična podela stavlja isti snimak i u trening i u test.** Uzastopni frejmovi istog videa su skoro identični, pa test skup sadrži „iste“ slike kao trening.

Klasa	Slika	Različitih snimaka
drugi_problem	737	9
varoa_grinje	1.051	5
zdrava	3.384	31

Mreža se trenira tako da što manje greši, i koristi bilo koju razliku između klasa koja joj u tome pomaže. Najlakša takva razlika u ovom skupu je pozadina, osvetljenje i podloga pojedinog snimka — ne izgled pčele.



Primeri slika po klasi u skupu BeeImage — klasa varoe ima prepoznatljivu tirkiznu podlogu, a grinja se ne vidi

Dokaz: test prečice

Da bi se hipoteza proverila, modelu su date samo četiri ugla svake test slike (25% širine i 25% visine). Pčela je u centru isečka, pa uglovi sadrže pozadinu. Kada bi model gledao pčelu, iz uglova bi pogađao na nivou slučaja (33% za tri klase).

Prava klasa	Slika	Cela slika — tačno	Samo uglovi — tačno
zdrava	507	96,4%	84,0%
varoa_grinje	158	97,5%	**73,9%**
drugi_problem	111	99,1%	10,8%

Iz komadića pozadine na kome pčele nema, model u 74% slučajeva „prepoznaje“ varou. Tačnost od 97% je dakle tačan broj, ali meri prepoznavanje snimka, a ne bolesti. Model nije upotrebljiv za stvarnu procenu zdravlja pčele.

Reprodukcija: `python -m src.shortcut_test`.

Pouke za glavni eksperiment

1. Skup mora imati **oznaku po pčeli**, dodeljenu na osnovu vidljivog obeležja.
2. Pozadina mora biti **ista za sve klase**.
3. Podela mora biti **po snimku**, tako da test sadrži snimke koje model nije video.
4. Visoka tačnost na test skupu nije dovoljna — treba **proveriti šta model gleda**, i testirati ga na slikama drugačijeg porekla.

Sva četiri zahteva su ugrađena u glavni eksperiment.

Skup podataka VarroaDataset

Opis i izvor skupa dati su u poglavlju „Pregled javnih skupova podataka“. Ovde je prikazana analiza preuzetog skupa (`python -m src.varroa_dataset`).

Raspodela po skupovima i klasama

Skup	Snimaka	Zdrava	Varoa (oznaka 1)	Varoa (oznaka 3)	Ukupno
train	6	5.671	1.974	580	8.225
val	5	1.425	373	78	1.876
test	2	2.466	736	206	3.408
ukupno	13	9.562	3.083	864	13.509

Odnos zdravih i zaraženih je 2,4 : 1. Oznaka 3 se tretira kao zaražena (obrazloženje u pregledu skupova).

Podela po snimku

Podelu na train/val/test dali su autori skupa, po snimku: **nijedan snimak se ne pojavljuje u dva skupa**. Svaki snimak sadrži i zdrave i zaražene pčele, pa pozadina snimka ne može da otkrije klasu — što je bio osnovni problem skupa Beelmage.

Snimak	Skup	Slika	Udeo zaraženih
2017-08-28_09-30-00-1_500_dirty_glass	train	1.040	41%
2017-08-28_16-31-55	train	5	100%
2017-08-30_15-42-59	train	2.020	17%
2017-09-20_19-24-55	train	3.839	38%
2017-09-25_16-03-38-2	train	915	23%
2017-10-17_16-41-10	train	406	30%
2017-08-28_16-32-55_30_sec	val	52	60%
2017-09-01_20-00-17	val	216	25%
2017-09-01_3-01-01	val	522	41%
2017-09-25_16-03-38	val	958	13%
2017-09-29_15-31-49	val	128	23%
2017-09-01_10-54-26	test	1.278	48%
2017-10-17_1-39-36	test	2.130	15%

Napomena: snimci `2017-09-25_16-03-38` (val) i `2017-09-25_16-03-38-2` (train) su verovatno dva dela iste sesije snimanja. To može blago da ulepša rezultate na validacionom skupu (koji služi samo za izbor checkpoint-a), ali ne utiče na test skup.

Grinja na slici

Svaka zaražena pčela ima 1–3 ručno označene grinje (3.332 sa 1, 549 sa 2, 66 sa 3). Srednja veličina označene grinje je 32×31 piksela na slici 160×280 — jasno vidljiva, za razliku od skupa Beelmage. Koordinate grinja se **ne koriste u treningu**; služe samo za proveru da li model gleda grinju.

Primeri iz validacionog skupa po oznaci. Crveni pravougaonik je nacrtan iz gt.csv radi prikaza — na samim slikama ga nema.

Transfer learning — pristup i primena

Zašto se mreža ne trenira od nule

Konvolucione neuronske mreže za klasifikaciju slika imaju milione do desetine miliona parametara i tipično zahtevaju stotine hiljada označenih primera. Trening skup ovog rada ima 8.225 slika. Treniranje od nule na tolikom skupu vodi u preprilagođavanje (*overfitting*): mreža zapamti trening slike umesto da nauči opšte obeležje.

Transfer learning rešava taj problem. Mreže korišćene u ovom radu prethodno su istrenirane na skupu **ImageNet** (oko 1,28 miliona slika, 1.000 klasa svakodnevnih objekata). Tokom tog treninga naučile su opšte vizuelne reprezentacije — niži slojevi prepoznaju ivice, uglove i boje, srednji teksture i ponavljajuće obrasce, viši delove objekata. Te reprezentacije nisu vezane za klase ImageNet-a i prenose se na nov domen, u ovom slučaju na fotografije pčela.

Mreža se konceptualno deli na dva dela:

Deo	Funkcija	Primer: ResNet50
Telo (*backbone*)	konvolucioni slojevi koji sliku pretvaraju u vektor obeležja	23,5 miliona parametara, izlaz 2.048 obeležja
Glava (*classifier*)	potpuno povezani sloj koji vektor obeležja preslikava u klase	2.048 → 1.000 klasa ImageNet-a

Tri arhitekture

Arhitektura	Parametara	Osnovna ideja	Težine (`torchvision`)
MobileNetV3-Large	4,2 M	dubinski razdvojive konvolucije i arhitektura pronađena automatskom pretragom (NAS), projektovana za mobilne uređaje	`MobileNet_V3_Large_Weights.IMAGENET1K_V2`
ResNet50	23,5 M	rezidualne veze koje omogućavaju treniranje veoma dubokih mreža; standardna referentna arhitektura	`ResNet50_Weights.IMAGENET1K_V2`
EfficientNetB0	4,0 M	uravnoteženo skaliranje dubine, širine i rezolucije	`EfficientNet_B0_Weights.IMAGENET1K_V1`

Primenjeni postupak

Postupak je identičan za sve tri arhitekture i sprovodi se u šest koraka. Implementacija: `src/models.py` i `src/train.py`.

1. Učitavanje prethodno istreniranog modela sa ImageNet težinama iz biblioteke `torchvision`.

2. Zamena završnog klasifikacionog sloja. Originalni sloj sa 1.000 izlaza zamenjuje se novim, slučajno inicijalizovanim, sa dva izlaza — `zdrava` i `varoa`. Mesto tog sloja razlikuje se po arhitekturi (`model.fc` kod ResNet-a, `model.classifier[-1]` kod MobileNetV3 i EfficientNetB0), što je u implementaciji sakriveno iza funkcije `build_model`.

3. Zamrzavanje prethodno naučenih slojeva. Svim parametrima tela postavlja se `requires_grad = False`, pa se trenira samo glava:

Arhitektura	Ukupno parametara	Trenira se u fazi 1	Trenira se u fazi 2
MobileNetV3-Large	4.204.594	1.232.642 (cela glava, dva sloja)	4.204.594 (sve)
ResNet50	23.512.130	4.098	23.512.130 (sve)
EfficientNetB0	4.010.110	2.562	4.010.110 (sve)

4. Treniranje nove glave (*feature extraction*): 3 epohe, optimizator AdamW, stopa učenja 10^{-3} sa kosinusnim opadanjem. Telo u ovoj fazi služi kao nepromenljiv izvlačilac obeležja. Cilj ove faze nije konačan model, nego da nova, slučajno inicijalizovana glava dobije razumne težine pre nego što se odmrzne telo — inače bi njeni veliki, nasumični gradijenti u prvim koracima fine-tuning-a pokvarili naučene reprezentacije tela.

5. Fine-tuning cele mreže. Kad se glava stabilizuje, odmrzava se cela mreža i trenira još 15 epoha sa stopom učenja 10^{-4} — deset puta manjom nego za glavu, uz kosinusno opadanje. Manja stopa je ključna: veliki koraci u ovoj fazi bi uništili prethodno naučene reprezentacije (*catastrophic forgetting*).

Odmrzavanje cele mreže, a ne samo poslednjeg bloka, rezultat je eksperimenta (poglavlje „Eksperimentalni rezultati“, odeljak „Izbor protokola treninga“). Obeležje koje model treba da prepozna — sitna, ovalna, crvenkasto-braon grinja od dvadesetak piksela — nije zastupljeno u ImageNet-u, i zamrznuti niži slojevi ga ne izdvajaju dovoljno dobro: sa odmrznutim samo poslednjim blokom model nije uspeo da nauči zadatak ni na trening skupu.

6. Izbor najboljeg checkpoint-a i evaluacija. Posle svake epohe model se proverava na validacionom skupu; čuva se stanje sa najboljim macro F1. Test skup se koristi tačno jednom, na kraju, i ni u jednom trenutku ne utiče na odluke o modelu.

Priprema slike (*letterbox*)

Slike u VarroaDataset-u su uspravne, 160×280 piksela. Standardna priprema slike za ove mreže (`weights.transforms()`) smanjuje kraću stranu na 232 piksela i iseca centralni kvadrat 224×224 — kod uspravne slike to bi odseklo oko 45% visine, upravo glavu i zadak pčele, gde je grinja često.

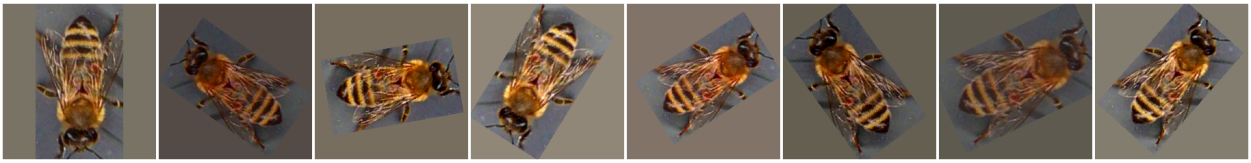
Zato se u ovom radu koristi **letterbox**: cela slika se smanji tako da duža strana bude 224 piksela, a ostatak kvadrata dopuni neutralnom bojom (srednja boja ImageNet-a, koja posle normalizacije daje vrednosti blizu nule). Ništa se ne odseca, a isti postupak radi i za fotografije korisnika bilo kog formata. Normalizacija po kanalu (srednja vrednost i standardna devijacija ImageNet-a) ostaje ona sa kojom su mreže trenirane.

Funkcija za pripremu slike napisana je **jednom** (`service/preprocessing.py`) i koriste je i trening i servis u produkciji. Time je isključeno da se slika u produkciji priprema drugačije nego u treningu — česta i teško uočljiva greška.

Augmentacija

Na trening skupu svaka slika se pri svakom prolazu nasumično menja, tako da mreža ne vidi dvaput istu sliku. Augmentacija je identična za sve tri arhitekture:

Transformacija	Opseg	Zašto
preslikavanje horizontalno i vertikalno	p = 0,5	pčela je snimljena odozgo, pa nema „gore“
rotacija	do ±180°	isto
pomeranje i promena razmere	±5%, 0,8–1,1	pčela nije uvek u centru i iste veličine
osvetljenje, kontrast, zasićenost	±30%	različita kamera i osvetljenje kod korisnika
nijansa (*hue*)	±2%	malo — crvenkasta boja grinje je signal
zamućenje (Gaussian blur)	p = 0,2	fotografije van fokusa



Augmentacija: original (levo) i sedam nasumičnih varijanti iste zaražene pčele

Disbalans klasa

U trening skupu je 5.671 zdrava i 2.554 zaražene pčele (2,2 : 1). Model koji bi uvek govorio „zdrava“ imao bi tačnost 69% bez ikakve upotrebne vrednosti. Primenjene su dve mere:

1. **Težinska funkcija greške** — `CrossEntropyLoss` sa težinama obrnuto proporcionalnim broju primera: `w_i = N / (K · n_i)`. Greška na zaraženoj pčeli košta 1,61, na zdravoj 0,73.
2. **Macro F1 kao kriterijum izbora** — checkpoint se bira po macro F1 na validacionom skupu, a ne po tačnosti, jer macro F1 jednako tretira obe klase.

Isti uslovi za sve tri arhitekture

Profesorica je tražila poređenje „pod što sličnijim uslovima“. Za sve tri arhitekture isti su: podela podataka, priprema slike, augmentacija, veličina batch-a (32), optimizator, stope učenja, broj epoha u obe faze, težine klasa, kriterijum izbora checkpoint-a i seme generatora slučajnih brojeva (42). Razlikuje se jedino telo mreže i, posledično, koji je njegov poslednji blok.

Izbor okvira: PyTorch ili TensorFlow/Keras

Oba okvira podržavaju sve tri tražene arhitekture sa ImageNet težinama i oba su u potpunosti dovoljna za ovaj rad. Odabran je **PyTorch sa `torchvision`** bibliotekom, iz tri razloga specifična za prirodu ovog zadatka.

1. Preprocesiranje je vezano za težine, ne za programera. U `torchvision`-u svaki objekat prethodno istreniranih težina nosi i metodu `transforms()` koja vraća tačno ono preprocesiranje sa kojim je model treniran. U Keras-u svaka arhitektura ima zasebnu funkciju `preprocess_input` sa različitim konvencijama — `EfficientNet` u Keras-u očekuje vrednosti piksela u opsegu 0–255 i normalizaciju izvodi interno, dok `ResNet50` očekuje već skalirane vrednosti. Zamena tih funkcija ne izaziva grešku u izvršavanju, samo tiho pogoršava rezultat. Pošto je cilj ovog rada **poređenje tri arhitekture pod što sličnijim uslovima**, okvir u kojem je ta klasa greške strukturno onemogućena ima direktnu metodološku prednost.

2. Jedna petlja treninga za sve tri arhitekture. U PyTorch-u se petlja treninga piše eksplicitno, pa su optimizator, stopa učenja, raspored opadanja, broj epoha, kriterijum izbora checkpointa i obrada disbalansa doslovno isti kod za sva tri modela — razlikuje se samo telo mreže. Eksplicitna petlja je ovde prednost, a ne režijski trošak: čini uslove eksperimenta proverljivim.

3. Čist put do produkcije. Model se izvozi u ONNX direktno iz PyTorch-a (`torch.onnx.export`), bez posrednog koraka. Iz Keras-a je potrebna dodatna konverzija alatom `tf2onnx`.

4. Ista priprema slike u treningu i produkciji. Transformacije u PyTorch-u su obične Python funkcije nad PIL slikama, pa je letterbox napisan jednom, bez zavisnosti od PyTorch-a, i ista funkcija se koristi u treningu i u servisu koji model izvršava. U Keras-u je priprema slike najčešće deo grafa modela ili `tf.data` pipeline-a, pa bi servis morao da zavisi od TensorFlow-a ili da je ponovi drugim kodom.

Dodatno, PyTorch ima nativnu podršku za **MPS** backend, što omogućava trening na grafičkom procesoru Apple Silicon računara na kojem je rad izveden, bez oslanjanja na eksterne GPU servise.

Verzije korišćene u radu: Python 3.12.15, PyTorch 2.14.1, torchvision 0.29.1, scikit-learn 1.9.1, onnxruntime 1.30.0.

Eksperimentalni rezultati

Izbor protokola treninga

Prvi pokušaj koristio je isti protokol kao eksperiment sa BeeImage-om (varijanta A): fine-tuning samo poslednjeg bloka tela, stopa učenja 10^{-5}. Model nije uspeo da nauči zadatak — tačnost i na samom trening skupu ostala je niska. Uzrok je to što zamrznuti slojevi, naučeni na ImageNet-u, ne izdvajaju sitno obeležje kao što je grinja. Na BeeImage-u to nije bio problem, jer je model tamo ionako učio pozadinu.

Protokol je zato podešen. Varijante su upoređene **isključivo na validacionom skupu** (snimci odvojeni i od treninga i od testa), na MobileNetV3 kao najbržoj arhitekturi. Izabrani protokol je zatim primenjen identično na sve tri arhitekture. Test skup nije korišćen ni u jednom koraku ovog izbora.

Varijanta	Arhitektura	Slika	Fine-tuning	Epohe (glava + FT)	LR fine-tuning	Train accuracy (kraj)	Najbolji val F1
A	MobileNetV3-Large	224 px	poslednji blok	15 + 10	1e-05	0,7590	0,6651
B	MobileNetV3-Large	224 px	cela mreža	3 + 15	0.0001	0,9274	0,9336
C	MobileNetV3-Large	320 px	cela mreža	3 + 15	0.0001	0,9346	0,9427

Odmrzavanje cele mreže (B) podiglo je val F1 sa 0,6651 na 0,9336. Veća slika (C) dala je još 0,009, što je manje od oscilacije val F1 između susednih epoha iste varijante (do 0,020), a trening je 4,7 puta sporiji (i inferencija sporija). Izabrana je varijanta **B**: fine-tuning cele mreže, slika 224×224.

Uslovi eksperimenta

Sve tri arhitekture trenirane su potpuno istim postupkom (poglavlje „Transfer learning“); razlikuje se samo telo mreže.

Parametar	Vrednost
Skup	VarroaDataset, zvanična podela po snimku
Trening / val / test	8.225 / 1.876 / 3.408 slika
Klase	zdrava, varoa
Priprema slike	letterbox 224×224, normalizacija ImageNet
Faza 1 (glava)	3 epoha, AdamW, lr 0.001, kosinusno opadanje
Faza 2 (fine-tuning)	15 epoha, lr 0.0001, cela mreža
Batch	32
Funkcija greške	CrossEntropy sa težinama klasa
Izbor checkpoint-a	najbolji macro F1 na val skupu
Seme	42
Uređaj	Apple M5, MPS

Poređenje arhitektura na test skupu

Precision, recall i F1 su *macro* proseci — prosek preko obe klase, bez obzira na broj primera. „Recall varoa“ je udeo zaraženih pčela koje je model prepoznao — najvažnija pojedinačna metrika za pčelara, jer je propuštena zaraza najskuplja greška. ROC AUC meri koliko dobro model razdvaja klase nezavisno od praga odluke (0,5).

Arhitektura	Accuracy	Precision	Recall	F1	Recall varoa	ROC AUC	Parametara	ONNX ms	Trening min
MobileNetV3-Large	0,9199	0,8964	0,9063	0,9011	0,8758	0,9662	4,2 M	2,6	20,4
ResNet50	0,9310	0,9152	0,9117	0,9134	0,8684	0,9753	23,5 M	11,9	40,0
EfficientNetB0	0,9217	0,9054	0,8970	0,9011	0,8418	0,9644	4,0 M	4,5	24,8

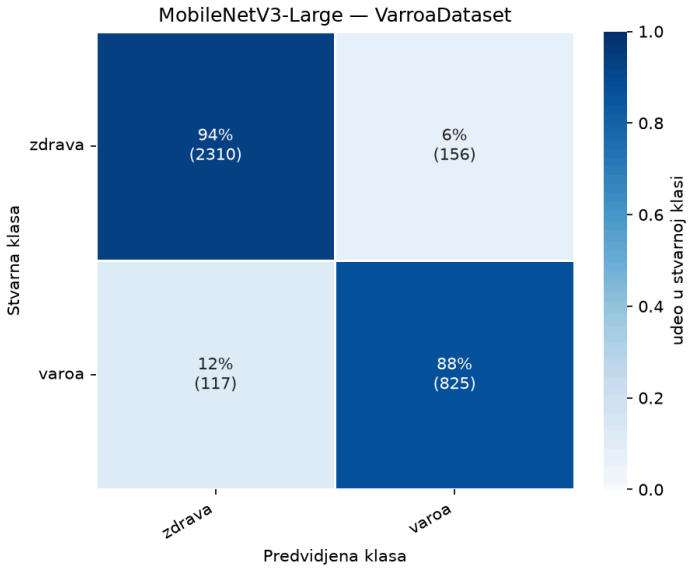
„ONNX ms“ je vreme obrade jedne slike u runtime-u koji se koristi u produkciji (onnxruntime, CPU, batch 1).

Metrike po klasi

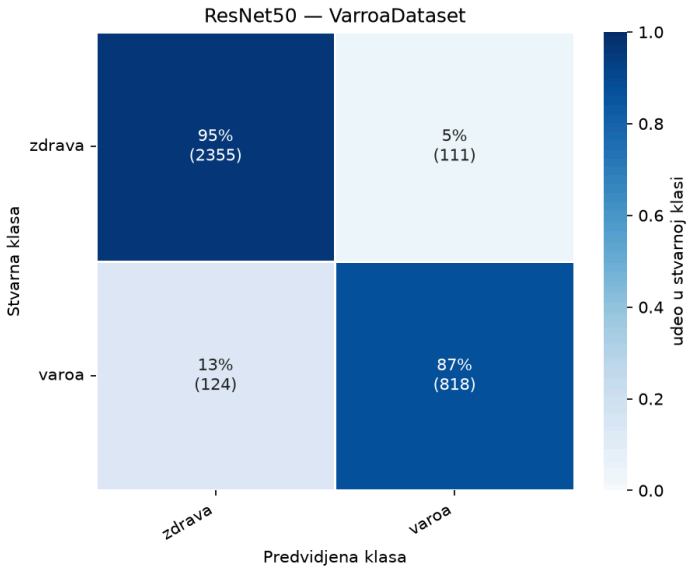
Arhitektura	Klasa	Precision	Recall	F1	Primera
MobileNetV3-Large	zdrava	0,9518	0,9367	0,9442	2.466
MobileNetV3-Large	varoa	0,8410	0,8758	0,8580	942
ResNet50	zdrava	0,9500	0,9550	0,9525	2.466
ResNet50	varoa	0,8805	0,8684	0,8744	942
EfficientNetB0	zdrava	0,9403	0,9521	0,9462	2.466
EfficientNetB0	varoa	0,8705	0,8418	0,8559	942

Matrice konfuzije

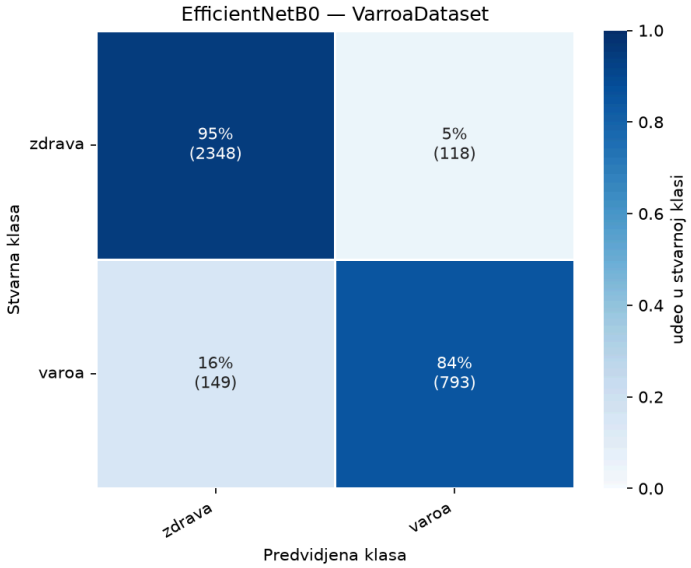
Arhitektura	Zdrava → zdrava	Zdrava → varoa (lažna uzbuna)	Varoa → zdrava (propuštena)	Varoa → varoa
MobileNetV3-Large	2.310	156	117	825
ResNet50	2.355	111	124	818
EfficientNetB0	2.348	118	149	793



Matrica konfuzije — MobileNetV3-Large



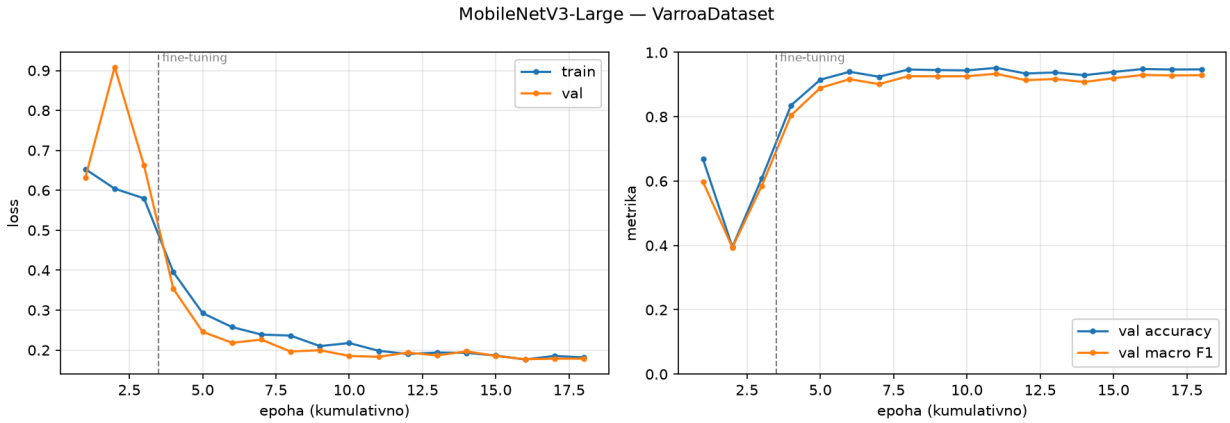
Matrica konfuzije — ResNet50



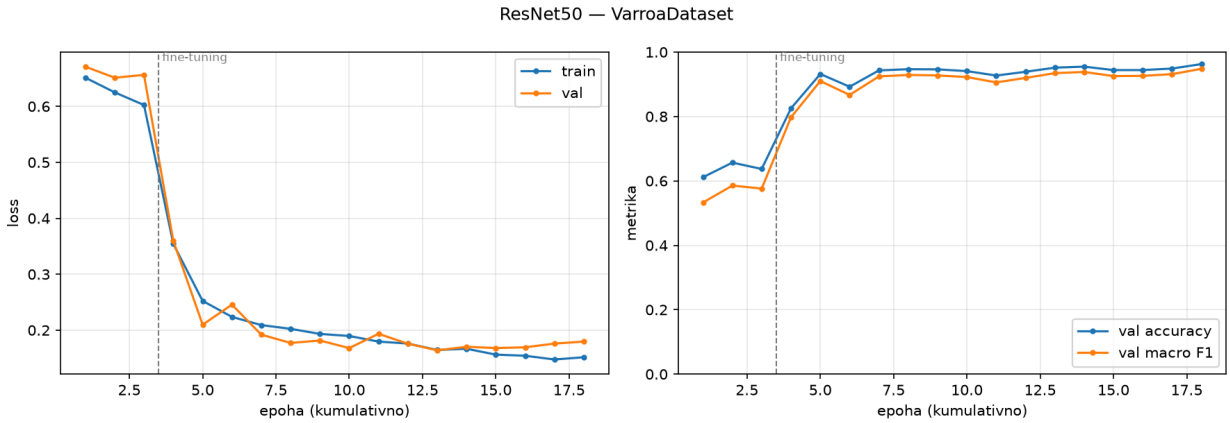
Matrica konfuzije — EfficientNetB0

Krive učenja

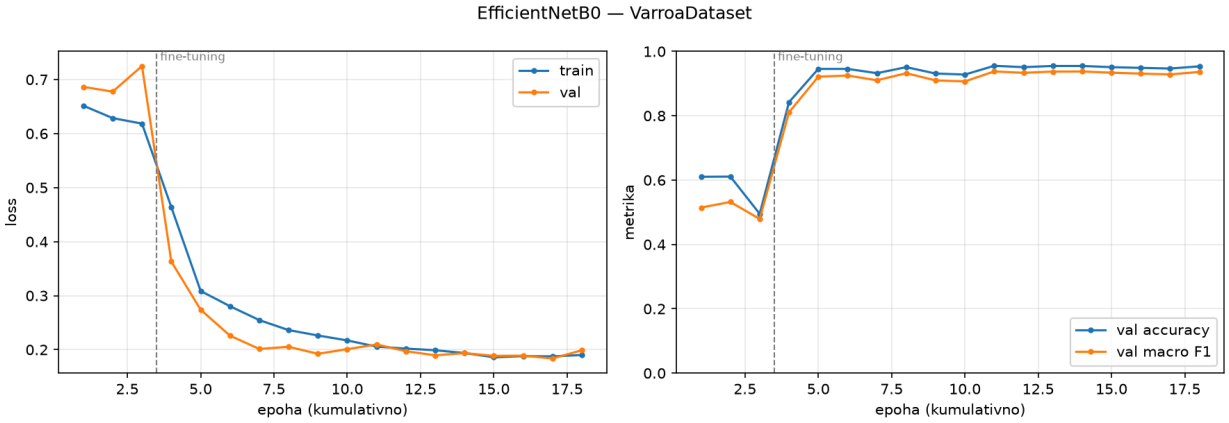
Vertikalna linija označava prelaz iz faze 1 (trenira se samo glava) u fine-tuning.



Krive učenja — MobileNetV3-Large



Krive učenja — ResNet50



Krive učenja — EfficientNetB0

Rezultat po test snimku

Test skup čine dva snimka sa vrlo različitim udelom zaraženih pčela. Ujednačen rezultat na oba ukazuje da model ne zavisi od uslova pojedinog snimka.

Arhitektura	2017-09-01_10-54-26 (48% zaraženih)	2017-10-17_1-39-36 (15% zaraženih)
MobileNetV3-Large	0,8826	0,9423
ResNet50	0,9061	0,9460
EfficientNetB0	0,8842	0,9441

Grinja vidljiva i delimično skrivena

Recall na varoi, odvojeno po originalnoj oznaci autora skupa.

Arhitektura	Recall — oznaka 1 (vidljiva)	Recall — oznaka 3 (delimično skrivena)
MobileNetV3-Large	0,9470	0,6214
ResNet50	0,9484	0,5825
EfficientNetB0	0,9239	0,5485

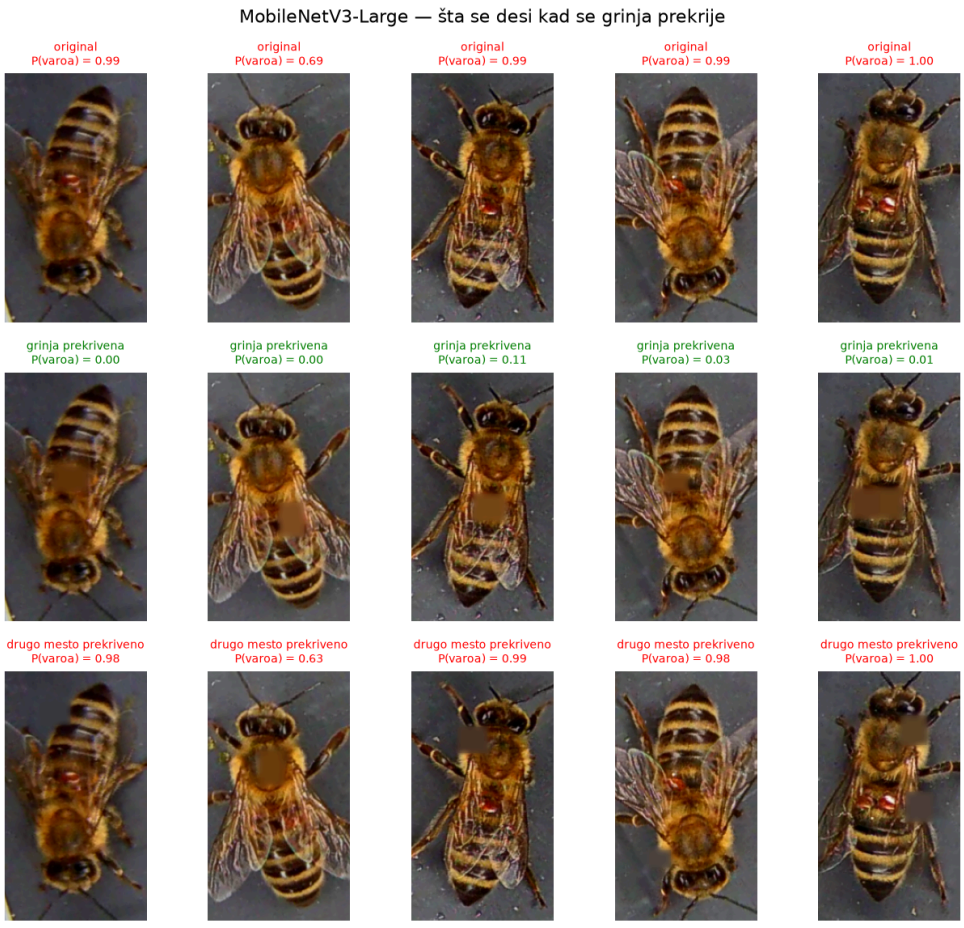
Da li model gleda grinju?

Prvi eksperiment je pokazao da visoka tačnost na test skupu nije dovoljna: model može postići 97% učeći pozadinu. Zato su za sve tri arhitekture izvedene tri dodatne provere ponašanja (`python -m src.model_checks`). Sve se rade na test skupu.

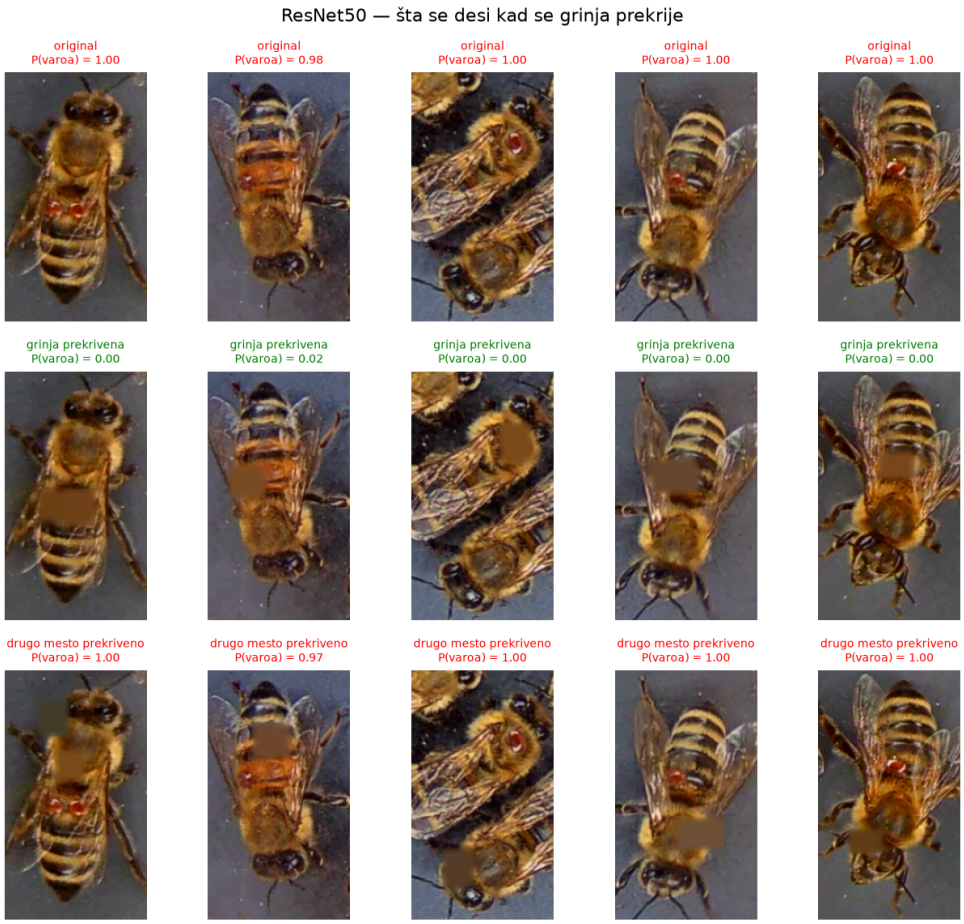
Provera 1: brisanje grinje

Na svakoj zaraženoj test pčeli koju je model prepoznao, svaka označena grinja se prekrije bojom svoje okoline. Ako odluka modela zavisi od grinje, procena treba da pređe u „zdrava“. Kao kontrola, isti broj jednako velikih površina prekrije se na drugom delu pčele, van grinje — to meri koliko sama intervencija (mrlja na slici) menja procenu. Razlika između ova dva broja je efekat grinje.

Arhitektura	Prepoznato (original)	Ostalo „varoa“ — grinja prekrivena	Ostalo „varoa“ — drugo mesto prekriveno	P(varoa): original → grinja → drugo mesto
MobileNetV3-Large	87,6%	3,5%	97,7%	0,94 → 0,07 → 0,92
ResNet50	86,8%	3,9%	98,2%	0,93 → 0,06 → 0,92
EfficientNetB0	84,2%	3,1%	95,8%	0,92 → 0,06 → 0,89



Primeri — MobileNetV3-Large: original, grinja prekrivena, drugo mesto prekriveno



Primeri — ResNet50: original, grinja prekrivena, drugo mesto prekriveno

Primeri — EfficientNetB0: original, grinja prekrivena, drugo mesto prekriveno

Provera 2: samo pozadina

Modelu se daju samo četiri ugla slike (25% × 25%), gde je pozadina. Pozadina je ista za obe klase, pa udeo odgovora „varoa“ treba da bude sličan za uglove zdravih i zaraženih pčela. Za poređenje: BeeImage model je iz uglova „prepoznao“ varou u 73,9% slučajeva.

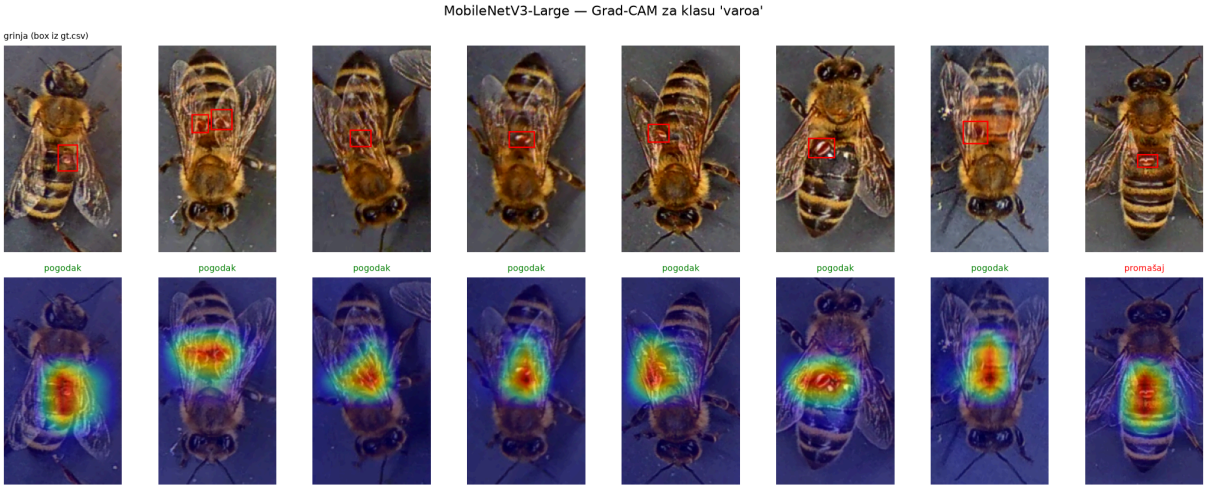
Arhitektura	„Varoa“ iz uglova zdravih pčela	„Varoa“ iz uglova zaraženih pčela
MobileNetV3-Large	2,8%	3,6%
ResNet50	0,7%	1,3%
EfficientNetB0	3,8%	5,3%

Provera 3: Grad-CAM

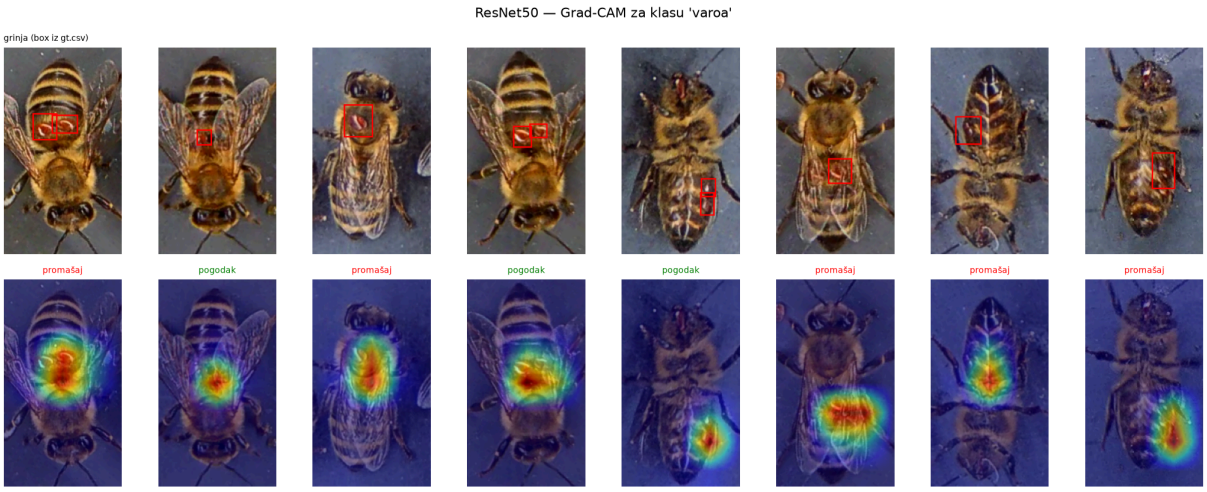
Grad-CAM je metoda koja pokazuje koji delovi slike su najviše doprineli odluci mreže za datu klasu: računa koliko promena svakog dela poslednjeg konvolucionog sloja menja izlaz za klasu „varoa“. Kao kvantitativna mera koristi se *pointing game*: da li najjača tačka mape pada u označeni pravougaonik grinje (uz toleranciju od 8 piksela, jer je Grad-CAM mapa gruba — 7×7 polja pre uvećanja).

Arhitektura	Uzoraka	Najjača tačka u grinji
MobileNetV3-Large	200	87,0%
ResNet50	200	46,5%
EfficientNetB0	200	87,0%

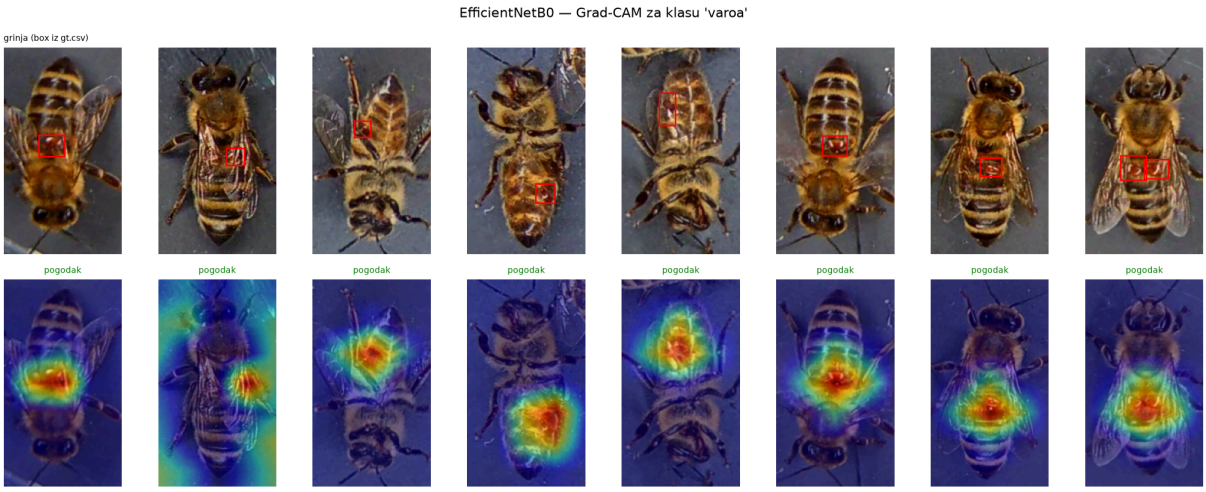
Grad-CAM i brisanje grinje mere različite stvari. Brisanje grinje je **uzročni** test: menja se ulaz i meri promena odluke. Grad-CAM je **opisna** mapa, gruba (7×7 polja) i zavisna od arhitekture. Kada model ima nizak rezultat u pointing game-u, a visok efekat u testu brisanja, to znači da je odluka zavisna od grinje, ali je mapa pažnje šira i obuhvata i okolni deo tela pčele — što se vidi na primerima ispod. Za zaključak „model gleda grinju“ presudan je test brisanja.



Grad-CAM — MobileNetV3-Large. Gornji red: grinja iz gt.csv; donji: mapa pažnje modela.



Grad-CAM — ResNet50. Gornji red: grinja iz gt.csv; donji: mapa pažnje modela.



Grad-CAM — EfficientNetB0. Gornji red: grinja iz gt.csv; donji: mapa pažnje modela.

Test van skupa za trening

Test skup VarroaDataset-a sadrži snimke koje model nije video, ali snimljene istom opremom, na istoj podlozi. Korisnik aplikacije će slati fotografije drugačijeg porekla. Zato su modeli testirani i na dva izvora slika koje nisu iz VarroaDataset-a (`python -m src.field_test`).

BeeDataset — druga kamera, druga podloga

7.490 slika pčela sa ulaza u košnicu (zelena podloga, druga kamera i rezolucija, Nemačka), od čega 1.266 sa oznakom varoe. Model ovaj skup nije video ni u jednoj fazi.

Arhitektura	Accuracy	Recall varoa	Precision varoa	F1 (macro)	ROC AUC
MobileNetV3-Large	0,8999	0,4755	0,8750	0,7793	0,9488
ResNet50	0,8794	0,3049	0,9438	0,6965	0,8694
EfficientNetB0	0,8650	0,2030	0,9923	0,6310	0,9279

Terenske fotografije

Fotografije pojedinačnih pčela sa proizvoljnom pozadinom (2), uključujući fotografiju koja je otkrila problem sa prvim modelom. Prikazana je verovatnoća klase „varoa“ (prag odluke 0,5). Folder `data/field_photos/` je pripremljen tako da se proširi novim fotografijama i test ponovi istom komandom.

Fotografija	Prava klasa	MobileNetV3-Large	ResNet50	EfficientNetB0
zdrava/pcela_na_cvetu.jpg	zdrava	0,106	0,005	0,042
varoa/makro_pcela_na_sacu_2_grinje.jpg	varoa	0,172	0,989	0,985

Izbor modela

Kriterijumi

Model se bira po tri grupe kriterijuma, ovim redom važnosti:

1. **Uspešnost na test skupu** — metrike koje je tražila profesorica: accuracy, precision, recall i F1, uz matricu konfuzije. Posebno se gleda recall na varoi (propuštena zaraza je najskuplja greška) i broj lažnih uzbuna.
2. **Da li model gleda grinju** i kako se ponaša na slikama drugačijeg porekla (BeeDataset, terenske fotografije).
3. **Inženjerski kriterijumi** — brzina inferencije i veličina modela u produkcionom okruženju.

Svi kriterijumi na jednom mestu

Arhitektura	Test F1 (macro)	Test recall varoa	Test ROC AUC	Grinja prekrivena → ostalo „varoa“	BeeDataset ROC AUC	BeeDataset recall varoa	ONNX ms/slika	Fajl MB
MobileNetV3-Large	0,9011	0,8758	0,9662	3,5%	0,9488	0,4755	2,6	17,1
ResNet50	0,9134	0,8684	0,9753	3,9%	0,8694	0,3049	11,9	94,2
EfficientNetB0	0,9011	0,8418	0,9644	3,1%	0,9279	0,2030	4,5	16,6

Odluka: ResNet50

Izabran je **ResNet50**.

Test skup. ResNet50 je najbolji po svim metrikama koje je profesorica navela — accuracy, precision, recall i F1 (macro) — kao i po ROC AUC. Ima najmanje lažnih uzbuna (zdrava pčela proglašena zaraženom): 111, naspram 156 kod MobileNetV3 i 118 kod EfficientNetB0. Recall na varoi je praktično isti kao kod MobileNetV3 (razlika je sedam pčela od 942).

Ponašanje. Sva tri modela prolaze test brisanja grinje: kada se grinja prekrije, procena „varoa“ ostaje kod svega 3–4% pčela, a kada se prekrije drugo mesto na pčeli — kod 96–98%. Odluka sva tri modela zavisi od grinje, a ne od pozadine. ResNet50 i EfficientNetB0 su ispravno prepoznali terensku fotografiju pčele sa dve grinje na saću — fotografiju na kojoj je prvi (BeeImage) model pogrešio — dok MobileNetV3 nije.

Brzina i veličina. ResNet50 je najsporiji (oko 12 ms po slici) i najveći (94 MB). Za ovu primenu to nije ograničenje: korisnik šalje jednu fotografiju i čeka odgovor preko mreže, pa je razlika između 3 i 12 ms neprimetna, a model se izvršava na serveru, ne na mobilnom uređaju.

Rezerve

Odluka nije jednoznačna i treba je čitati uz sledeće:

- **Razlike na test skupu su male** — oko jednog procentnog poena — a svaka arhitektura je trenirana jednom. Bez ponavljanja sa više semena ne može se tvrditi da je razlika statistički značajna.
- **MobileNetV3 se bolje prenosi na BeeDataset** (ROC AUC 0,95 naspram 0,87 kod ResNet50), a 4,6 puta je brži i 5,5 puta manji. Ako bi se model izvršavao na mobilnom uređaju, ili ako bi buduće fotografije više ličile na snimke sa ulaza u košnicu nego na makro fotografije, MobileNetV3 bi bio bolji izbor.
- **Terenski test ima malo fotografija**; on ilustruje, ali ne dokazuje.

Arhitektura sistema dozvoljava zamenu modela bez izmena u backendu i frontendu (poglavlje „Integracija“), pa se odluka može promeniti kada terenski test skup bude veći. Svi modeli su izvezeni: `models/varroa_*.onnx`.

Poređenje sa prvim eksperimentom

	BeeImage model	**VarroaDataset model (ResNet50)**
Tačnost na svom test skupu	97%	93%
Test skup	isti snimci kao trening	snimci koje model nije video
Iz same pozadine prepoznaje varou	74% slučajeva	~1%
Grinja prekrivena → i dalje „varoa“	nije merljivo	3,9%
Terenska fotografija sa dve grinje	„zdrava“ 99,97%	„varoa“ 98,9%

Novi model ima nižu tačnost na papiru, ali je ona **poštena**: meri prepoznavanje grinje na novim snimcima, a ne pamćenje snimaka iz treninga.

Integracija u informacioni sistem „Košnica“

Postojeći sistem

Sloj	Tehnologija
Frontend	Angular 17, samostalne (*standalone*) komponente, TypeScript
Backend	ASP.NET Core (.NET 10), REST API, JWT autentifikacija
Baza	PostgreSQL, Entity Framework Core
Spoljne integracije	OpenWeatherMap

Arhitektura rešenja

Model se ne izvršava unutar .NET procesa, nego u zasebnom **Python servisu (FastAPI)**. ASP.NET backend ostaje jedina tačka koju frontend poznaje i jedini sloj koji pristupa bazi i proverava ovlašćenja.

```
Angular — modal košnice, tab „Zdravlje pčela“
   │  POST /api/hives/{hiveId}/health-checks   (multipart: slika, JWT)
   ▼
ASP.NET Core — BeeHealthChecksController
   │  1. provera da korisnik ima pravo na košnicu
   │  2. provera fajla (format, veličina)
   │  3. BeeHealthService ──HTTP POST /predict──► FastAPI servis (Python)
   │                                              onnxruntime + bee_health.onnx
   │                                              priprema slike po bee_health.meta.json
   │                      ◄── {label, confidence, all_probs, model_version}
   │  4. tek ako je analiza uspela: slika se snima (LocalStorageService)
   │  5. upis u tabelu BeeHealthChecks — istorija pregleda
   ▼
JSON odgovor → Angular prikazuje procenu, verovatnoće i istoriju
```

Zašto zaseban Python servis

Razmatrane su dve opcije.

Opcija A — ONNX Runtime unutar .NET-a (`Microsoft.ML.OnnxRuntime`). Prednost: jedan proces i jedan deployment. Nedostatak: priprema slike morala bi se ponovo napisati u C#. Svako odstupanje od pripreme iz treninga tiho pogoršava predikcije, bez ikakve greške u izvršavanju.

Opcija B — Python/FastAPI servis (izabrano). Servis koristi istu funkciju za pripremu slike kao trening (`service/preprocessing.py`), sa parametrima pročitanim iz `bee_health.meta.json` koji nastaje pri izvozu modela. Rizik tihog odstupanja je time uklonjen. Servis se može nezavisno skalirati, a model zameniti novom verzijom bez ponovnog objavljivanja .NET aplikacije — upravo to je u ovom radu i urađeno pri prelasku sa BeeImage na VarroaDataset model.

Model se izvozi u format **ONNX**, pa servis koristi `onnxruntime` umesto PyTorch-a: servis ne zavisi od biblioteke za trening, a Docker slika je reda veličine 100 MB umesto nekoliko gigabajta.

Python servis (FastAPI)

FastAPI je Python biblioteka za izradu REST API-ja, po ulozi slična ASP.NET Minimal API-ju. Servis (`KosnicaML/service/`) ima dve rute:

Ruta	Opis
`GET /health`	status, verzija modela, arhitektura, klase
`POST /predict`	prima sliku (multipart polje `file`), vraća procenu

Primer odgovora:

```
{
  "label": "varoa",
  "confidence": 0.9731,
  "all_probs": {"zdrava": 0.0269, "varoa": 0.9731},
  "model_version": "bee_health"
}
```

Model se učitava jednom, pri pokretanju servisa. Fajl koji nije slika vraća grešku 400, fajl veći od 10 MB grešku 413.

Provera ispravnosti. Skripta `service/verify_against_pytorch.py` propušta ceo test skup kroz servis (ONNX + priprema slike servisa) i kroz originalni PyTorch model iz treninga, i poredi verovatnoće za svaku sliku.

Model	Test slika	Najveća razlika u verovatnoći	Različitih predikcija	Accuracy kroz servis
ResNet50	3.408	0,00005	0	0,9310

Backend (ASP.NET Core)

Komponenta	Opis
`Services/IBeeHealthService.cs`, `BeeHealthService.cs`	HTTP klijent ka FastAPI servisu; adresa iz promenljive okruženja `ML_SERVICE_URL`, inače `MlService:BaseUrl` iz `appsettings.json`
`Services/LocalStorageService.cs`	snima fajlove u `wwwroot/uploads/` i vraća njihov URL; zamenio je ranije postojeći `MockStorageService`, koji nije čuvao fajlove
`Models/BeeHealthCheck.cs`	nova tabela: košnica, URL slike, predviđena klasa, verovatnoća, verovatnoće svih klasa (`jsonb`), verzija modela, vreme, korisnik
`Controllers/BeeHealthChecksController.cs`	rute za analizu, istoriju i brisanje
EF migracija `AddBeeHealthChecks`	primenjuje se automatski pri pokretanju

Ruta	Opis	Ovlašćenje
`POST /api/hives/{hiveId}/health-checks`	analiza fotografije	svaki član tima pčelara
`GET /api/hives/{hiveId}/health-checks`	istorija pregleda, najnovije prvo	svaki član tima
`DELETE /api/hives/{hiveId}/health-checks/{id}`	brisanje pregleda i slike	samo pčelar (uloga *Beekeeper*)

Ponašanje u greškama: ako ML servis nije dostupan, backend vraća 503 sa porukom, a slika se ne snima. Pristup tuđoj košnici vraća 404 (ne otkriva se ni da košnica postoji). Fajl koji nije slika vraća 400.

Svi navedeni slučajevi su provereni stvarnim HTTP zahtevima nad pokrenutim sistemom.

Frontend (Angular)

U modalu detalja košnice dodat je treći tab, **„Zdravlje pčela“** (komponenta `hive-health-check`, servis `bee-health.service.ts`):

- izbor fotografije i pregled pre slanja
- rezultat: oznaka u boji (zelena — zdrava, crvena — varoa), verovatnoća procene i trake verovatnoća za svaku klasu
- istorija pregleda košnice sa sličicom, procenom, datumom i korisnikom
- brisanje pregleda uz potvrdu
- uputstvo korisniku: jedna pčela, izbliza, po mogućstvu odozgo

Zamena modela

Model se menja zamenom dva fajla — `models/bee_health.onnx` i `models/bee_health.meta.json` — i ponovnim pokretanjem servisa. Backend i frontend ne zavise od broja ni naziva klasa: backend čuva verovatnoće kao JSON objekat, a frontend prikazuje klase koje model vrati. Stari zapisi u istoriji (iz prvog, BeeImage modela, sa tri klase) i dalje se ispravno prikazuju.

Ograničenja i dalji rad

Ograničenja skupa podataka

Jedno okruženje snimanja. Sve slike VarroaDataset-a snimljene su istom opremom, odozgo, na istoj sivoj podlozi. Model je zato video samo jednu vrstu pozadine i osvetljenja. Augmentacija (rotacija, osvetljenje, boja, zamućenje) ublažava, ali ne rešava ovo ograničenje. Koliko se model prenosi na drugačije snimke meri se na skupu BeeDataset i terenskim fotografijama (poglavlje „Test van skupa za trening“).

Dve klase. Model razlikuje samo zdravu pčelu i pčelu sa vidljivom varoom. Druga zdravstvena stanja (deformisana krila, druge bolesti, odsustvo matice) nisu obuhvaćena, jer nijedan pronađeni javni skup nema pouzdane oznake za njih po pojedinačnoj pčeli.

Vidljiva grinja nije isto što i zaražena košnica. Model procenjuje da li se na fotografisanoj pčeli vidi grinja. Pčela bez vidljive grinje može poticati iz zaražene košnice — grinja može biti ispod krila, između segmenata ili u leglu. Za procenu stanja košnice potrebno je analizirati više pčela; pojedinačna procena je pomoćna informacija pčelaru, a ne dijagnoza.

Nedokumentovana oznaka. Oznaka 3 u skupu nije opisana u dokumentaciji. U radu se tretira kao zaražena pčela, u skladu sa zvaničnom statistikom autora; rezultati su prikazani i odvojeno za oznake 1 i 3.

Moguće preklapanje val i train snimka. Dva snimka iz iste sesije (25.09.2017) nalaze se jedan u trening, a drugi u validacionom skupu. To može blago ulepšati validacione rezultate, na osnovu kojih se bira checkpoint i protokol, ali ne utiče na test skup.

Ograničenja eksperimenta

Jedno seme. Svaka arhitektura trenirana je jednom, sa semenom 42. Razlike između arhitektura manje od oko jednog procentnog poena nisu nužno značajne — za pouzdaniji zaključak trening bi trebalo ponoviti sa više semena i prikazati srednju vrednost i rasipanje. Trening na MPS backend-u ni sa fiksnim semenom nije potpuno deterministič an.

Protokol biran na jednoj arhitekturi. Protokol treninga (fine-tuning cele mreže, stopa učenja, veličina slike) izabran je na validacionom skupu sa MobileNetV3, pa primenjen na sve tri. Druga arhitektura bi mogla imati koristi od drugačijih parametara; izabran je jedan zajednički protokol da bi poređenje bilo pod istim uslovima, kako je traženo.

Mali uzorak terenskih fotografija. Terenski test ima malo fotografija i služi kao ilustracija, ne kao statistički pouzdana mera. Folder `data/field_photos/` je pripremljen tako da se proširi i test ponovi.

Grad-CAM je gruba mapa. Poslednji konvolucioni sloj ima prostornu rezoluciju 7×7 (odnosno 10×10 za veće ulaze), pa mapa pokazuje područje, ne tačnu tačku. Zato je u merenju korišćena tolerancija, a glavni dokaz da model gleda grinju je test brisanja grinje, ne Grad-CAM.

Ograničenja integracije

Skladištenje slika je lokalno. Fotografije se čuvaju na disku servera (`wwwroot/uploads/`). Za produkciju na platformi sa netrajnim diskom (npr. Render) potrebno je trajno skladište (npr. Cloudinary ili S3) — interfejs `IStorageService` je za to već pripremljen.

Docker slika servisa nije testirana na mašini na kojoj je rad izveden (Docker nije instaliran); servis je testiran lokalno.

Dalji rad

1. **Proširenje terenskog test skupa** fotografijama iz stvarnih pčelinjaka i ponovna procena. Ako se pokaže pad na drugačijim pozadinama: augmentacija zamenom pozadine (izdvajanje pčele sa sive podloge i postavljanje na raznovrsne pozadine) ili dodavanje BeeDataset-a u trening.
2. **Detekcija umesto klasifikacije.** Koordinate grinja u VarroaDataset-u omogućavaju treniranje detektora (npr. YOLO) koji pronalazi i broji grinje na fotografiji sa više pčela — bliže stvarnom pregledu okvira.
3. **Procena na nivou košnice.** Agregacija procena više pčela iz iste košnice u procenu stepena zaraze, uz praćenje kroz vreme u istoriji pregleda.
4. **Automatska intervencija.** Pri proceni varoe sa visokom verovatnoćom sistem može predložiti intervenciju tipa „Tretman“ sa statusom „Planirano“, koja se pojavljuje u postojećem planeru pčelara.
5. **Kalibracija praga odluke.** Prag 0,5 može se pomeriti ka većoj osetljivosti (više otkrivenih zaraza po cenu više lažnih uzbuna), u zavisnosti od toga šta je pčelaru skuplje.

Prilog: reprodukcija rezultata

Svi rezultati u izveštaju mogu se ponovo dobiti komandama ispod. Tabele u ovom dokumentu generišu se automatski iz rezultata eksperimenta — nijedan broj nije upisan ručno.

Okruženje

Komponenta	Verzija
Python	3.12.15
PyTorch / torchvision	2.14.1 / 0.29.1
scikit-learn	1.9.1
onnxruntime	1.30.0
Hardver	Apple M5, PyTorch MPS backend

```
cd KosnicaML
python3.12 -m venv .venv
./.venv/bin/pip install -r requirements.txt
```

Glavni eksperiment (VarroaDataset)

```
./run_varroa.sh
```

Skripta redom: preuzima VarroaDataset sa Zenodo-a (~1,2 GB), trenira tri arhitekture, evaluira ih na test skupu, izvodi provere ponašanja i test van skupa za trening. Trajanje na Apple M5: oko sat i po.

Korak	Komanda	Rezultat
preuzimanje i statistika	`python -m src.varroa_dataset`	`data/varroa/`
trening	`python -m src.train --arch <arh> --dataset varroa`	`runs/varroa_<arh>/best.pt`, `history.csv`
evaluacija	`python -m src.evaluate --all --dataset varroa`	`metrics.json`, matrica konfuzije, krive učenja
provere ponašanja	`python -m src.model_checks --all`	`checks.json`, `gradcam.png`, `masking_examples.png`
test van skupa	`python -m src.field_test`	`runs/field_test.json`
izvoz u ONNX	`python -m src.export_onnx --run <run>`	`models/<ime>.onnx` + `.meta.json`
latencija	`python -m src.benchmark_onnx --pattern "varroa_*.onnx" --out latency_benchmark_varroa.csv`	CSV
provera servisa	`python -m service.verify_against_pytorch`	`models/verification.json`
izveštaj	`python -m src.make_final_report`	`report/izvestaj_final.{md,docx,pdf}`

BeeDataset za test van skupa preuzima se sa `https://raspbee.de/BeeDataset_20201121.zip` u `data/beedataset/`.

Prvi eksperiment (BeeImage)

```
./run_all.sh                    # preuzimanje, 6 treninga, evaluacija
python -m src.shortcut_test     # test prečice
```

Notebook-ovi

U folderu `notebooks/`, u pet koraka. Ne treniraju ništa — čitaju gotove rezultate, crtaju grafike i čuvaju ih u `grafici/`:

Notebook	Sadržaj
`01_dataseti.ipynb`	pregled skupova, BeeImage (snimci po klasi, curenje), VarroaDataset
`02_prvi_eksperiment_beeimage.ipynb`	rezultati BeeImage modela, ručni test, test prečice
`03_transfer_learning_i_trening.ipynb`	transfer learning, letterbox, augmentacija, izbor protokola, krive učenja
`04_evaluacija_i_poredjenje.ipynb`	metrike, matrice konfuzije, ROC krive, analiza grešaka
`05_provera_i_izbor_modela.ipynb`	brisanje grinje, pozadina, Grad-CAM, BeeDataset, terenske fotografije, izbor

Generišu se iz `notebooks/build_notebooks.py`. Prezentacija za konsultacije: `python -m src.make_prezentacija` → `report/prezentacija_konsultacije.pdf`.

Pokretanje aplikacije

```
# 1. ML servis
cd KosnicaML && ./.venv/bin/uvicorn service.main:app --port 8000
# 2. backend (http://localhost:5264)
cd KosnicaApi && dotnet watch
# 3. frontend (http://localhost:4200)
cd KosnicaUI && npm start
```